C# Assignments

Events and Delegates

- **Assignment 1: Notification System**
  - Build a Stock class that raises an event when its price changes.
  - Use delegates to notify multiple subscribers (e.g., EmailNotifier, SMSNotifier).
  - Demonstrate subscribing/unsubscribing dynamically.

Collections

- **Assignment 2: Task Management System**
  - Use List<T> to store tasks.
  - Use Stack<T> to implement an undo feature (e.g., undo last added task).
  - Use Queue<T> to process tasks in FIFO order.
  - Use Dictionary<int, string> to map task IDs to descriptions.

Polymorphism

- **Assignment 3: Payment Gateway**
  - Create a base class Payment.
  - Derive classes: CreditCardPayment, UPIPayment, NetBankingPayment.
  - Override a method ProcessPayment() to show different implementations.
  - Demonstrate runtime polymorphism by calling the method via base reference.

Inheritance

- **Assignment 4: Employee Hierarchy**
  - Base class: Employee (properties: Name, Salary).
  - Derived classes: Manager, Developer, Intern.
  - Add role-specific methods (e.g., ApproveLeave(), WriteCode()).
  - Show how inheritance reduces redundancy.

Combined Assignment (All Concepts)

- **Assignment 5: Library Management System**
  - Use Inheritance: Book, EBook, AudioBook.
  - Use Polymorphism: Override DisplayDetails() in each type.
  - Use Collections: Store books in List<Book>, track borrowed books in Queue<Book>, maintain user-book mapping in Dictionary<User, List<Book>>.
  - Use Events/Delegates: Raise an event when a book is borrowed, notify subscribers (e.g., EmailNotifier).
  - Demonstrate all concepts working together.

Async/Await Assignments

- **Assignment 6: Weather Data Fetcher**
  - Create a method GetWeatherAsync() that fetches mock weather data.
  - Await multiple tasks (e.g., temperature, humidity, wind speed).
  - Combine results into a single report.

- **Assignment 7: Order Processing System**
    - Simulate placing an order asynchronously.
    - Use await to process payment, update inventory, and send confirmation email.
    - Demonstrate exception handling with try/catch in async methods.